

```

#include <Wire.h>
#include <MPU6050.h>
#include <ESP32Servo.h>
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

void enviarMensaje(String msg);
void procesarComando(String cmd);
void procesarSet(String param);
void pararTerapia();
void activarEmergencia(String origen);
bool verificarEmergencia();
float obtenerAnguloMuneca();
float obtenerAnguloAntebrazo();

// PINES
#define PIN_SERVO_FLEX1 25 // SG90 - Flexión dedo 1
#define PIN_SERVO_FLEX2 26 // SG90 - Flexión dedo 2
#define PIN_SERVO_EXT 27 // SG90 - Extensión dedos
#define PIN_SERVO_MUNECA 14 // MG996R - Extensión muñeca
#define PIN_VIBRADOR 5 // Motor de vibración (ambos en paralelo vía Q1)
#define PIN_EMERGENCY_SW 35 // Switch físico de emergencia (INPUT_PULLUP)

// DIRECCIONES I2C MPU6050
#define IMU_MUNECA 0x68 // U1: AD0 = GND
#define IMU_ANTEBRAZO 0x69 // U3: AD0 = VCC

// BLE UUIDs (compatibles con MIT App Inventor)
#define SERVICE_UUID "12345678-1234-1234-1234-123456789abc"
#define CHAR_RX_UUID "12345678-1234-1234-1234-123456789ab1" // App → ESP32
#define CHAR_TX_UUID "12345678-1234-1234-1234-123456789ab2" // ESP32 → App

// PARÁMETROS DE TERAPIA (defaults)
struct TerapiaParams {
    float anguloObjetivoMuneca = 30.0; // grados (0=neutro, + = flexión)
    float anguloObjetivoAntebrazo = 15.0; // grados tolerancia antebrazo
    float toleranciaAngulo = 10.0; // ± grados para aceptar posición
    int repeticiones = 10; // ciclos de flexión/extensión
    int pwmFuerzaFlex = 180; // ángulo máx servo flexión (0-180)
};

```

```

    int    pwmFuerzaExt          = 0;          // ángulo máx servo extensión
(0-180)
    int    velocidadMs          = 15;         // ms entre cada paso de servo
(suavidad)
    int    pausaEntreCiclosMs   = 1000;       // ms de pausa entre ciclos
};

// VARIABLES GLOBALES
MPU6050    imuMuneca(IMU_MUNECA);
MPU6050    imuAntebrazo(IMU_ANTEBRAZO);
Servo      servoFlex1, servoFlex2, servoExt, servoMuneca;
TerapiaParams params;

BLEServer*    pServer          = nullptr;
BLECharacteristic* pCharTX      = nullptr;
bool          deviceConnected  = false;

// Estados del sistema
enum Estado {
    ESPERA,          // Esperando conexión BLE o calibración
    CALIBRANDO,      // Terapeuta posicionando al paciente
    POSICION_LISTA,  // Posición verificada, esperando inicio
    TERAPIA_ACTIVA,  // Ciclos de terapia en curso
    EMERGENCIA       // Parada de emergencia activa
};

Estado estadoActual = ESPERA;

int    repeticionActual = 0;
bool    emergenciaActiva = false;
bool    stopPorApp      = false;

// BLE CALLBACKS
class ServerCallbacks : public BLEServerCallbacks {
    void onConnect(BLEServer* pSvr) override {
        deviceConnected = true;
        Serial.println("[BLE] Cliente conectado");
        enviarMensaje("CONECTADO:AURA_v1.0");
    }
    void onDisconnect(BLEServer* pSvr) override {
        deviceConnected = false;
        Serial.println("[BLE] Cliente desconectado");
        // Parar terapia si se desconecta
        if (estadoActual == TERAPIA_ACTIVA) {
            pararTerapia();
        }
    }
}

```

```

        pSvr->startAdvertising();
    }
};

// Procesa comandos recibidos desde la app
class RXCallbacks : public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic* pChar) override {
        String valor = pChar->getValue().c_str();
        Serial.println("[BLE RX] " + valor);
        procesarComando(valor);
    }
};

// SETUP
void setup() {
    Serial.begin(115200);
    Serial.println("=== AURA Sistema de Rehabilitación ===");

    // Pin de emergencia
    pinMode(PIN_EMERGENCY_SW, INPUT_PULLUP);
    pinMode(PIN_VIBRADOR, OUTPUT);
    digitalWrite(PIN_VIBRADOR, LOW);

    // I2C para IMUs
    Wire.begin();

    // Inicializar IMU Muñeca
    imuMuneca.initialize();
    if (imuMuneca.testConnection()) {
        Serial.println("[IMU] Muñeca (0x68) OK");
    } else {
        Serial.println("[IMU] ERROR: Muñeca no detectada");
    }

    // Inicializar IMU Antebrazo
    imuAntebrazo.initialize();
    if (imuAntebrazo.testConnection()) {
        Serial.println("[IMU] Antebrazo (0x69) OK");
    } else {
        Serial.println("[IMU] ERROR: Antebrazo no detectado");
    }

    // Servos
    ESP32PWM::allocateTimer(0);
    ESP32PWM::allocateTimer(1);
}

```

```

ESP32PWM::allocateTimer(2);
ESP32PWM::allocateTimer(3);

servoFlex1.setPeriodHertz(50);
servoFlex2.setPeriodHertz(50);
servoExt.setPeriodHertz(50);
servoMuneca.setPeriodHertz(50);

servoFlex1.attach(PIN_SERVO_FLEX1, 500, 2400);
servoFlex2.attach(PIN_SERVO_FLEX2, 500, 2400);
servoExt.attach(PIN_SERVO_EXT, 500, 2400);
servoMuneca.attach(PIN_SERVO_MUNECA, 500, 2400);

// Posición inicial (neutra)
servoFlex1.write(90);
servoFlex2.write(90);
servoExt.write(90);
servoMuneca.write(90);

// BLE
iniciarBLE();

estadoActual = ESPERA;
Serial.println("[SISTEMA] Listo. Esperando conexión BLE...");
}

// LOOP PRINCIPAL
void loop() {
    // — PRIORIDAD 1: Verificar switch de emergencia físico —
    if (digitalRead(PIN_EMERGENCY_SW) == LOW) {
        if (!emergenciaActiva) {
            activarEmergencia("SW_FISICO");
        }
        delay(50);
        return; // No hacer nada más mientras esté pulsado
    } else {
        if (emergenciaActiva) {
            // Switch liberado → reset
            emergenciaActiva = false;
            stopPorApp = false;
            estadoActual = ESPERA;
            Serial.println("[EMERGENCIA] Switch liberado. Sistema en espera.");
            enviarMensaje("EMERGENCIA:LIBERADA");
        }
    }
}

```

```

// — PRIORIDAD 2: Stop por app —
if (stopPorApp) {
    pararTerapia();
    stopPorApp = false;
    return;
}

// — Máquina de estados —
switch (estadoActual) {

    case ESPERA:
        // No hace nada, espera comandos BLE
        delay(200);
        break;

    case CALIBRANDO: {
        // Lee IMUs y envía ángulos a la app cada 200ms
        float angMuneca = obtenerAnguloMuneca();
        float angAntebrazo = obtenerAnguloAntebrazo();

        // Verificar si la posición ya coincide con el objetivo
        bool munecaOk = abs(angMuneca - params.anguloObjetivoMuneca)
<= params.toleranciaAngulo;
        bool antebrazoOk = abs(angAntebrazo - params.anguloObjetivoAntebrazo)
<= params.toleranciaAngulo;

        // Enviar datos a la app
        String datos = "ANGULOS:" + String(angMuneca, 1) + ":" +
String(angAntebrazo, 1)
            + ":M" + (munecaOk ? "1" : "0")
            + ":A" + (antebrazoOk ? "1" : "0");
        enviarMensaje(datos);

        if (munecaOk && antebrazoOk) {
            // ¡Posición correcta! Vibrar para indicar al paciente
            vibrarConfirmacion();
            estadoActual = POSICION_LISTA;
            enviarMensaje("ESTADO:POSICION_LISTA");
            Serial.println("[ESTADO] Posición verificada. Listo para iniciar
terapia.");
        }
        delay(200);
        break;
    }
}

```

```

    case POSICION_LISTA:
        // Espera comando START desde la app
        delay(200);
        break;

    case TERAPIA_ACTIVADA:
        ejecutarCicloTerapia();
        break;

    case EMERGENCIA:
        // Espera a que se libere el switch o se envíe RESET desde app
        delay(100);
        break;
}
}

// LECTURA DE IMUs

// Devuelve el ángulo de flexión/extensión de la muñeca en grados
// Usa el eje X del acelerómetro del IMU U1
float obtenerAnguloMuñeca() {
    int16_t ax, ay, az, gx, gy, gz;
    imuMuñeca.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
    // Ángulo de inclinación sobre eje X (pitch)
    float angulo = atan2((float)ay, (float)az) * 180.0 / PI;
    return angulo;
}

// Devuelve el ángulo de estabilidad del antebrazo
float obtenerAnguloAntebrazo() {
    int16_t ax, ay, az, gx, gy, gz;
    imuAntebrazo.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
    float angulo = atan2((float)ay, (float)az) * 180.0 / PI;
    return angulo;
}

// CICLO DE TERAPIA
void ejecutarCicloTerapia() {
    if (repeticionActual >= params.repeticiones) {
        finalizarTerapia();
        return;
    }
}

```

```

    Serial.println("[TERAPIA] Ciclo " + String(repeticionActual + 1) + "/" +
String(params.repeticiones));
    enviarMensaje("CICLO:" + String(repeticionActual + 1) + ":" +
String(params.repeticiones));

    // — FASE 1: FLEXIÓN —
    if (!verificarEmergencia()) return;
    enviarMensaje("FASE:FLEXION");
    moverServosFlexion();

    // Verificar que se logró la flexión y vibrar
    if (!verificarEmergencia()) return;
    vibrarPulso(1); // 1 pulso = flexión completa

    delay(params.pausaEntreCiclosMs / 2);

    // — FASE 2: EXTENSIÓN —
    if (!verificarEmergencia()) return;
    enviarMensaje("FASE:EXTENSION");
    moverServosExtension();

    // Verificar extensión completa y vibrar (retroalimentación propioceptiva)
    if (!verificarEmergencia()) return;
    vibrarPulso(2); // 2 pulsos = extensión completa

    delay(params.pausaEntreCiclosMs);

    repeticionActual++;
    enviarMensaje("REP_OK:" + String(repeticionActual));
}

// MOVIMIENTO DE SERVOS (con rampa suave)

// Mueve un servo suavemente de posición actual a destino
void moverServoSuave(Servo &servo, int destino) {
    if (!verificarEmergencia()) return;
    int actual = servo.read();
    int paso = (destino > actual) ? 1 : -1;
    while (actual != destino) {
        if (!verificarEmergencia()) return;
        actual += paso;
        servo.write(actual);
        delay(params.velocidadMs);
    }
}

```

```

void moverServosFlexion() {
    // SG90_1 y SG90_2 hacen flexión simultánea
    // SG90_3 va a posición neutra (no extiende)
    // MG996R hace flexión de muñeca

    // Movimiento simultáneo simplificado (paso a paso todos juntos)
    int pos1    = servoFlex1.read();
    int pos2    = servoFlex2.read();
    int pos3    = servoExt.read();
    int posMun  = servoMuneca.read();

    int dest1   = params.pwmFuerzaFlex; // flexión dedos
    int dest2   = params.pwmFuerzaFlex;
    int dest3   = 90;                  // neutro para extensión
    int destMun = params.pwmFuerzaFlex; // flexión muñeca

    int maxPasos = max({abs(dest1-pos1), abs(dest2-pos2),
                       abs(dest3-pos3), abs(destMun-posMun)});

    for (int i = 0; i <= maxPasos; i++) {
        if (!verificarEmergencia()) return;
        if (abs(dest1 - pos1) > 0) pos1 += (dest1 > pos1) ? 1 : -1;
        if (abs(dest2 - pos2) > 0) pos2 += (dest2 > pos2) ? 1 : -1;
        if (abs(dest3 - pos3) > 0) pos3 += (dest3 > pos3) ? 1 : -1;
        if (abs(destMun - posMun) > 0) posMun += (destMun > posMun) ? 1 : -1;
        servoFlex1.write(pos1);
        servoFlex2.write(pos2);
        servoExt.write(pos3);
        servoMuneca.write(posMun);
        delay(params.velocidadMs);
    }
}

void moverServosExtension() {
    // SG90_3 hace extensión, SG90_1 y SG90_2 van a neutro
    // MG996R vuelve a posición neutra (extensión muñeca)

    int pos1    = servoFlex1.read();
    int pos2    = servoFlex2.read();
    int pos3    = servoExt.read();
    int posMun  = servoMuneca.read();

    int dest1   = 90;                  // neutro dedos
    int dest2   = 90;

```



```

int dest3    = params.pwmFuerzaExt;    // extensión dedos
int destMun = 90;                      // neutro muñeca

int maxPasos = max({abs(dest1-pos1), abs(dest2-pos2),
                    abs(dest3-pos3), abs(destMun-posMun)});

for (int i = 0; i <= maxPasos; i++) {
    if (!verificarEmergencia()) return;
    if (abs(dest1 - pos1) > 0) pos1 += (dest1 > pos1) ? 1 : -1;
    if (abs(dest2 - pos2) > 0) pos2 += (dest2 > pos2) ? 1 : -1;
    if (abs(dest3 - pos3) > 0) pos3 += (dest3 > pos3) ? 1 : -1;
    if (abs(destMun - posMun) > 0) posMun += (destMun > posMun) ? 1 : -1;
    servoFlex1.write(pos1);
    servoFlex2.write(pos2);
    servoExt.write(pos3);
    servoMuneca.write(posMun);
    delay(params.velocidadMs);
}
}

// VIBRACIÓN
// Vibración de confirmación de posición correcta (largo)
void vibrarConfirmacion() {
    digitalWrite(PIN_VIBRADOR, HIGH);
    delay(600);
    digitalWrite(PIN_VIBRADOR, LOW);
    delay(200);
    digitalWrite(PIN_VIBRADOR, HIGH);
    delay(600);
    digitalWrite(PIN_VIBRADOR, LOW);
}

// N pulsos cortos para retroalimentación propioceptiva
void vibrarPulso(int n) {
    for (int i = 0; i < n; i++) {
        digitalWrite(PIN_VIBRADOR, HIGH);
        delay(250);
        digitalWrite(PIN_VIBRADOR, LOW);
        if (i < n - 1) delay(150);
    }
}

// EMERGENCIA Y PARADA
bool verificarEmergencia() {
    if (digitalRead(PIN_EMERGENCY_SW) == LOW || stopPorApp) {

```

```

        activarEmergencia(stopPorApp ? "APP_STOP" : "SW_FISICO");
        return false;
    }
    return true;
}

void activarEmergencia(String origen) {
    emergenciaActiva = true;
    estadoActual      = EMERGENCIA;
    // Detener todo inmediatamente
    servoFlex1.write(90);
    servoFlex2.write(90);
    servoExt.write(90);
    servoMuneca.write(90);
    digitalWrite(PIN_VIBRADOR, LOW);
    Serial.println("[EMERGENCIA] Activada por: " + origen);
    enviarMensaje("EMERGENCIA:" + origen);
}

void pararTerapia() {
    estadoActual = ESPERA;
    servoFlex1.write(90);
    servoFlex2.write(90);
    servoExt.write(90);
    servoMuneca.write(90);
    digitalWrite(PIN_VIBRADOR, LOW);
    repeticionActual = 0;
    enviarMensaje("ESTADO:DETENIDO");
    Serial.println("[TERAPIA] Detenida.");
}

void finalizarTerapia() {
    estadoActual = ESPERA;
    repeticionActual = 0;
    // Vibración de finalización: 3 pulsos largos
    for (int i = 0; i < 3; i++) {
        digitalWrite(PIN_VIBRADOR, HIGH);
        delay(400);
        digitalWrite(PIN_VIBRADOR, LOW);
        delay(200);
    }
    enviarMensaje("ESTADO:TERAPIA_COMPLETA");
    Serial.println("[TERAPIA] ¡Completada exitosamente!");
}

```

```
void procesarComando(String cmd) {
    cmd.trim();

    if (cmd == "INICIAR_CALIBRACION") {
        estadoActual = CALIBRANDO;
        enviarMensaje("ESTADO:CALIBRANDO");
    } else if (cmd == "GUARDAR_ANGULO") {
        // Guarda la posición actual como el objetivo
        params.anguloObjetivoMuneca = obtenerAnguloMuneca();
        params.anguloObjetivoAntebrazo = obtenerAnguloAntebrazo();
        enviarMensaje("ANGULO_GUARDADO:" + String(params.anguloObjetivoMuneca, 1)
            + ":" + String(params.anguloObjetivoAntebrazo, 1));
    } else if (cmd == "INICIAR_TERAPIA") {
        if (estadoActual == POSICION_LISTA || estadoActual == ESPERA) {
            repeticionActual = 0;
            estadoActual = TERAPIA_ACTIVADA;
            enviarMensaje("ESTADO:TERAPIA_ACTIVADA");
        } else {
            enviarMensaje("ERROR:POSICION_NO_VERIFICADA");
        }
    }

    } else if (cmd == "STOP") {
        stopPorApp = true;
    }

    } else if (cmd == "RESET") {
        emergenciaActiva = false;
        stopPorApp = false;
        repeticionActual = 0;
        estadoActual = ESPERA;
        servoFlex1.write(90);
        servoFlex2.write(90);
        servoExt.write(90);
        servoMuneca.write(90);
        digitalWrite(PIN_VIBRADOR, LOW);
        enviarMensaje("ESTADO:RESET_OK");
    }

    } else if (cmd == "GET:PARAMS") {
        String p = "PARAMS:"
            + String(params.anguloObjetivoMuneca, 1) + ":"
            + String(params.anguloObjetivoAntebrazo, 1) + ":"
            + String(params.repeticiones) + ":"
            + String(params.pwmFuerzaFlex) + ":"
            + String(params.pwmFuerzaExt) + ":"
    }
}
```

```

        + String(params.toleranciaAngulo, 1);
    enviarMensaje(p);

} else if (cmd.startsWith("SET:")) {
    procesarSet(cmd.substring(4));
}
}

void procesarSet(String param) {
    if (param.startsWith("ANG_MUN:")) {
        params.anguloObjetivoMuneca = param.substring(8).toFloat();
        enviarMensaje("OK:ANG_MUN:" + String(params.anguloObjetivoMuneca, 1));
    } else if (param.startsWith("ANG_ANT:")) {
        params.anguloObjetivoAntebrazo = param.substring(8).toFloat();
        enviarMensaje("OK:ANG_ANT:" + String(params.anguloObjetivoAntebrazo, 1));
    } else if (param.startsWith("REPS:")) {
        params.repeticiones = param.substring(5).toInt();
        enviarMensaje("OK:REPS:" + String(params.repeticiones));
    } else if (param.startsWith("PWM_FLEX:")) {
        params.pwmFuerzaFlex = constrain(param.substring(9).toInt(), 0, 180);
        enviarMensaje("OK:PWM_FLEX:" + String(params.pwmFuerzaFlex));
    } else if (param.startsWith("PWM_EXT:")) {
        params.pwmFuerzaExt = constrain(param.substring(8).toInt(), 0, 180);
        enviarMensaje("OK:PWM_EXT:" + String(params.pwmFuerzaExt));
    } else if (param.startsWith("TOL:")) {
        params.toleranciaAngulo = param.substring(4).toFloat();
        enviarMensaje("OK:TOL:" + String(params.toleranciaAngulo, 1));
    }
}

// BLE - INICIALIZACIÓN Y ENVÍO
void iniciarBLE() {
    BLEDevice::init("AURA_REHAB");
    pServer = BLEDevice::createServer();
    pServer->setCallbacks(new ServerCallbacks());

    BLEService* pService = pServer->createService(SERVICE_UUID);

    // Característica TX (notificaciones al celular)
    pCharTX = pService->createCharacteristic(

```

```

        CHAR_TX_UUID,
        BLECharacteristic::PROPERTY_NOTIFY
    );
    pCharTX->addDescriptor(new BLE2902());

    // Característica RX (recibe comandos del celular)
    BLECharacteristic* pCharRX = pService->createCharacteristic(
        CHAR_RX_UUID,
        BLECharacteristic::PROPERTY_WRITE | BLECharacteristic::PROPERTY_WRITE_NR
    );
    pCharRX->setCallbacks(new RXCallbacks());

    pService->start();
    pServer->getAdvertising()->start();
    Serial.println("[BLE] Advertising iniciado como 'AURA_REHAB'");
}

void enviarMensaje(String msg) {
    if (deviceConnected && pCharTX) {
        pCharTX->setValue(msg.c_str());
        pCharTX->notify();
        Serial.println("[BLE TX] " + msg);
    }
}

```